

Image Recognition



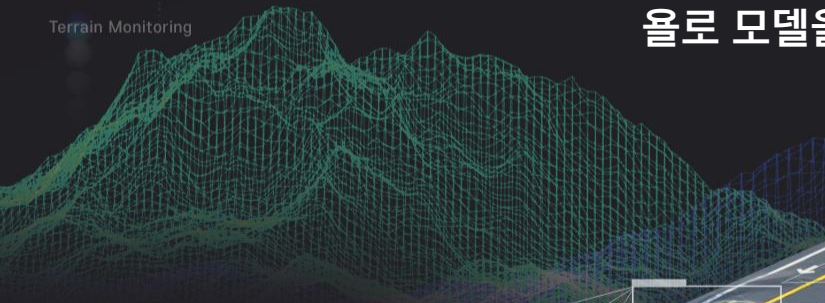
강원혁신플랫폼

컴퓨터 비전



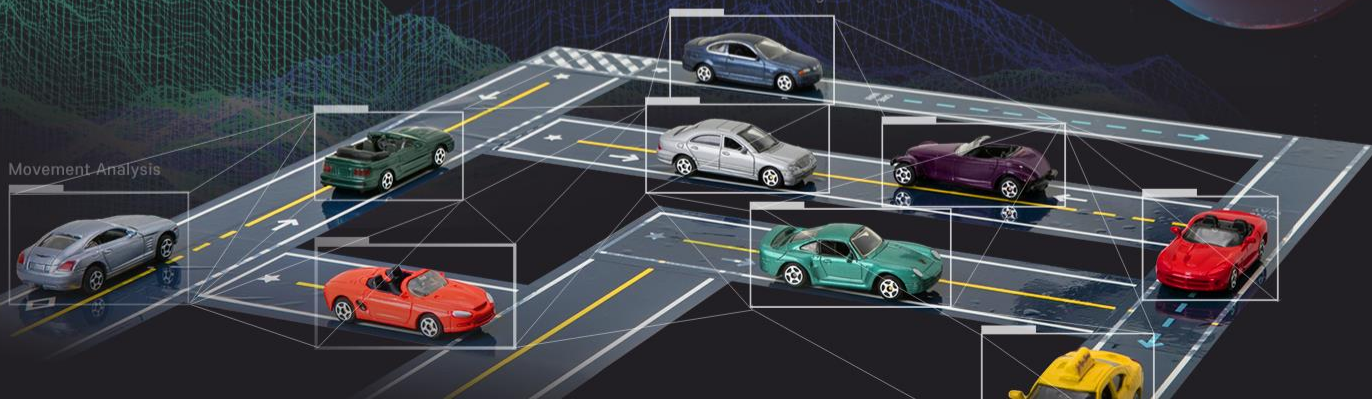
올로 모델을 이용한 이미지 객체 인식

Terrain Monitoring



Autonomous Driving

Movement Analysis



■ YOLO(You Only Look Once)는 Real-Time 객체 검출임

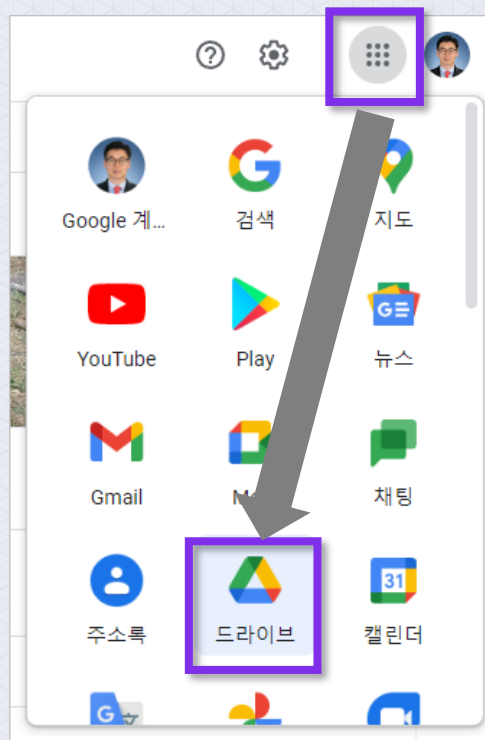
🔄 YOLO v3을 이용해 이미지에서 객체 검출을 해보자



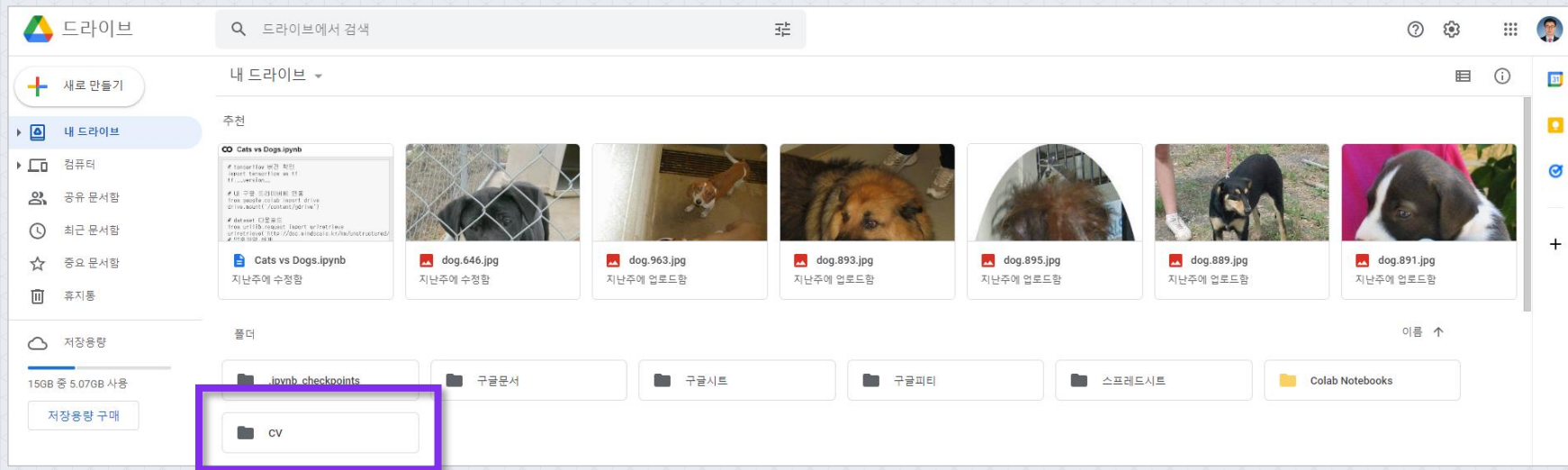
```
indexes: [ 9  4 26 27  1 21 19 24]  
1 bus 0.944  
4 car 0.983  
9 car 0.994  
19 person 0.630  
21 person 0.916  
24 car 0.543  
26 car 0.978  
27 car 0.974
```



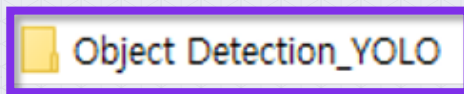
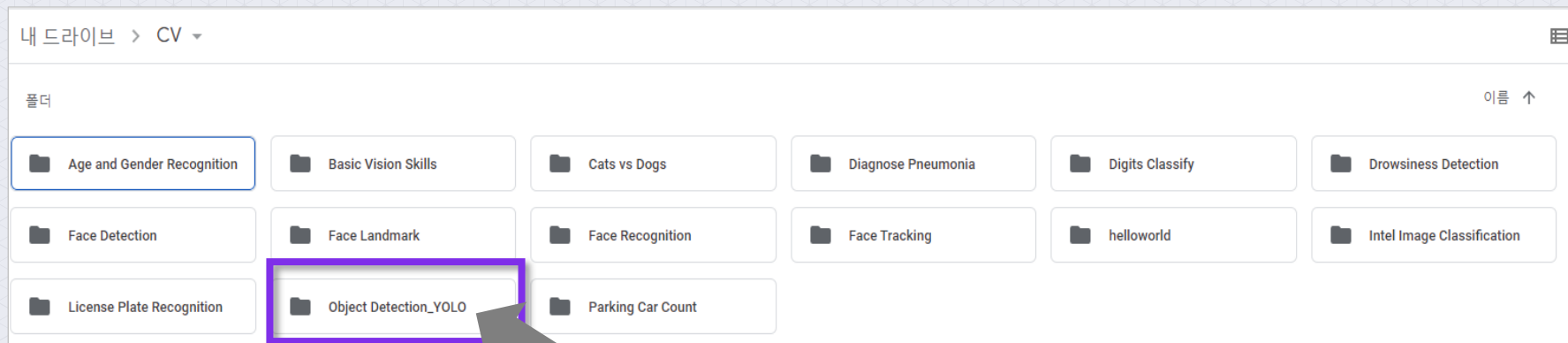
- 먼저, 구글 Colab을 이용해 보자
- 구글 크롬 브라우저에서 아래 그림과 같이 **구글 드라이브**를 클릭



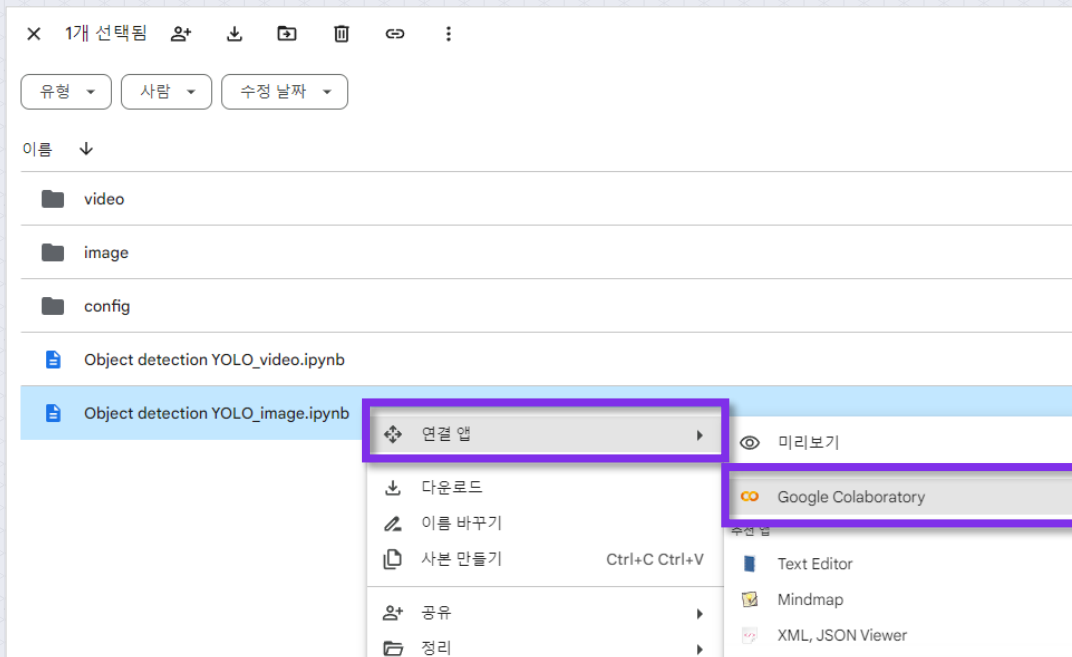
구글 드라이브에서 CV 폴더를 클릭



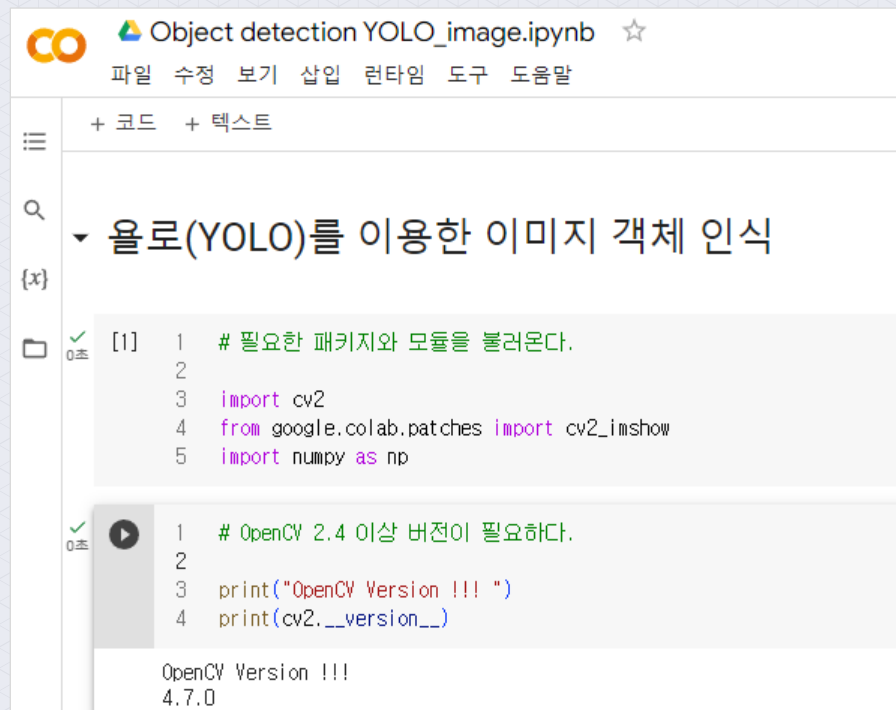
- 구글 드라이브 > CV > Object Detection_YOLO 폴더를 클릭
- 강의 실습 자료를 다운로드하여 [Object Detection_YOLO] 폴더를 구글 드라이브 CV 폴더 아래에 복사(업로드)



- 아래의 그림의 Object Detection_YOLO 폴더에서 Object detection YOLO_image.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭함



- 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것



Object detection YOLO_image.ipynb ☆

파일 수정 보기 삽입 런타임 도구 도움말

+ 코드 + 텍스트

올로(YOLO)를 이용한 이미지 객체 인식

```
[1] 1 # 필요한 패키지와 모듈을 불러온다.  
2  
3 import cv2  
4 from google.colab.patches import cv2_imshow  
5 import numpy as np
```

```
1 # OpenCV 2.4 이상 버전이 필요하다.  
2  
3 print("OpenCV Version !!! ")  
4 print(cv2.__version__)
```

OpenCV Version !!!
4.7.0

- 필요한 패키지와 모듈을 불러옴
- 구글 Colab 노트북 화면에서 실행 버튼 클릭

```
# 필요한 패키지와 모듈을 불러온다.  
  
import cv2  
from google.colab.patches import cv2_imshow  
import numpy as np
```


■ OpenCV의 버전을 확인해보자

- 🔄 구글 Colab 노트북 화면에서 실행 버튼 클릭

```
1  # OpenCV 2.4 이상 버전이 필요하다.  
2  
3  print("OpenCV Version !!! ")  
4  print(cv2.__version__)
```

📄 OpenCV Version !!!
4.7.0

- 🔄 YOLOv3을 이용한 객체 검출은 OpenCV 3.4 이상 버전이 필요

- ① 내 구글 드라이브에 연동
- ② 구글 Colab 노트북 화면에서 실행 버튼 클릭



- ③ [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동
- ④ gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있음

구글 Colab 화면에서 내 구글 드라이브 연동

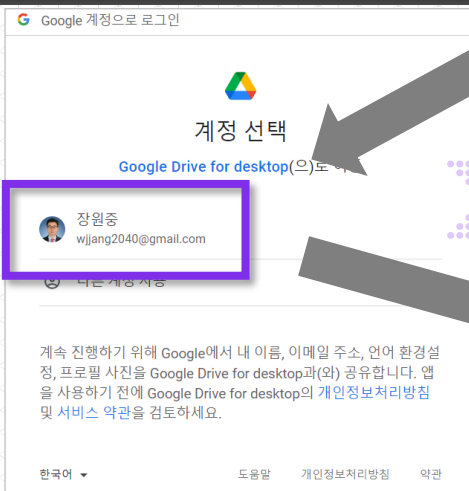
- [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용]
- 아래 화면을 참고하여 **gdrive**에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

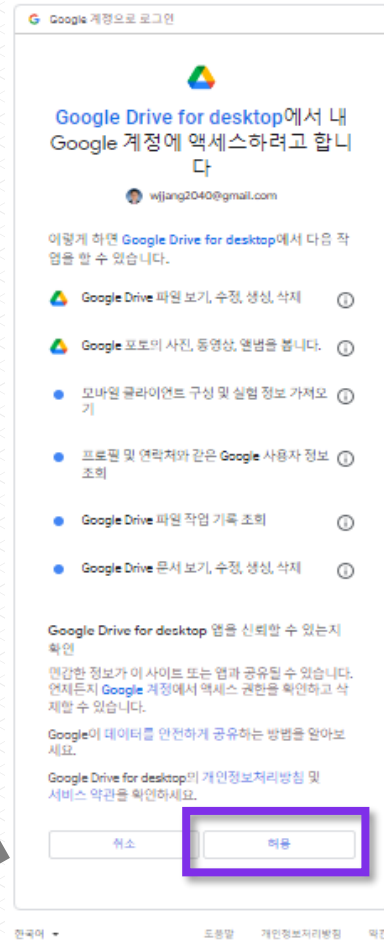
이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니오

Google Drive에 연결



구글 드라이버 접근 허용



■ 옰로 모델에 필요한 파일 준비

- YOLOv3의 가중치 저장파일 다운로드
- YOLO를 이용해 객체 검출을 하기 위해서는 몇 가지 파일을 다운로드 해야 함

```
# yolov3.weights 파일 다운로드

from urllib.request import urlretrieve

urlretrieve('https://pjreddie.com/media/files/yolov3.weights', 'yolov3.weights')

('yolov3.weights', <http.client.HTTPMessage at 0x7f65ca54a190>)
```

- yolov3.weights : YOLOv3의 가중치를 저장한 파일
- 이 파일은 YOLO 공식 사이트인 <https://pjreddie.com/media/file/yolov3.weights>에서 다운로드 함

VM에서 yolov3.weights 파일을 확인

```
# 구글 드라이브로 다운로드된 'yolov3.weights' 파일 확인

!!ls

gdrive sample_data yolov3.weights
```

내 구글 드라이브의 아래 위치로 yolov3.weights 파일을 이동

“구글 드라이브 > CV > Object Detection_YOLO > config >” 위치로 이동

```
1 # 내 구글 드라이브로 yolov3.weights 파일 이동
2
3 !mv yolov3.weights gdrive/My Drive/CV/Object Detection_YOLO/config/
4 print('yolov3.weights move complete!!')

yolov3.weights move complete!!
```

■ YOLOv3-tiny의 가중치 저장파일 다운로드

- 🔄 YOLO를 이용해 객체 검출을 하기 위해서는 몇 가지 파일을 다운로드 해야 함

```
1 # YOLOv3-tiny 가중치 파일
2 # - yolov3-tiny.weights 파일 다운로드
3
4 from urllib.request import urretrieve
5
6 urretrieve('https://pjreddie.com/media/files/yolov3-tiny.weights', 'yolov3-tiny.weights')
```

- 🔄 yolov3-tiny.weights : YOLOv3-tiny의 가중치를 저장한 파일
- 🔄 이 파일은 YOLO 공식 사이트인 <https://pjreddie.com/media/file/yolov3-tiny.weights> 에서 다운로드 함

- VM에서 yolov3-tiny.weights 파일을 확인

```
1 # 구글 드라이브로 다운로드된 'yolov3-tiny.weights' 파일 확인
2
3 !ls
```

gdrive sample_data yolov3-tiny.weights

- 내 구글 드라이브의 아래 위치로 yolov3-tiny.weights 파일을 이동

“구글 드라이브 > CV > Object Detection_YOLO > config >” 위치로 이동

```
1 # 내 구글 드라이브로 yolov3-tiny.weights 파일 이동
2
3 !mv yolov3-tiny.weights gdrive/My Drive/CV/Object Detection_YOLO/config/
4 print('yolov3-tiny.weights move complete!!')
```

yolov3-tiny.weights move complete!!

참고

YOLOv3 vs. YOLOv3-tiny

컴퓨터 비전에서 작은 객체 검출은 제한된 해상도와 정보로 검출하기 어렵다

🔄 이 문제를 해결하기 위해 기존 방법의 대부분은 높은 정확도 향상을 위해 속도를 희생함

정확도

YOLOv3 > YOLOv3-tiny

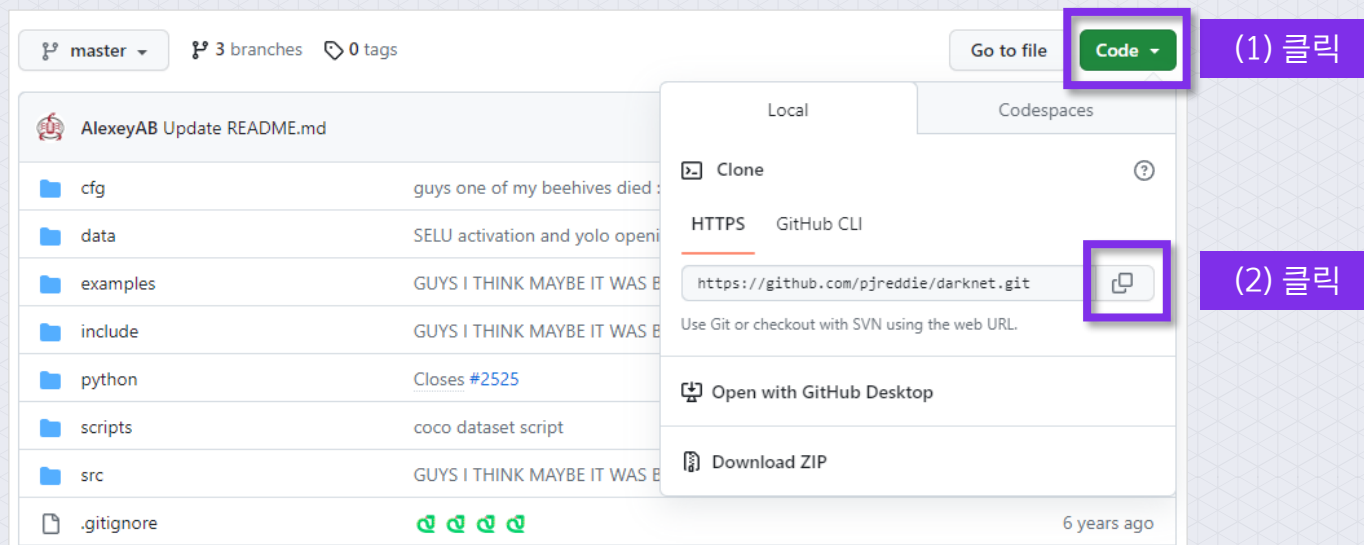
처리 속도

YOLOv3-tiny > YOLOv3

- 🔄 YOLOv3-tiny는 속도가 빠른 대신 인식 정확도는 YOLOv3가 더 높음
- 🔄 반면, 임베디드 같은 낮은 사양의 환경에서 YOLOv3-tiny를 이용하는게 좋을 것

■ darknet에서 옰로 신경망 구조(.cfg)와 부류명(coco.names) 파일을 다운로드

- 🕒 GitHub에 있는 darknet의 dataset를 다운로드 받도록 함
- 🕒 URL <https://github.com/pjreddie/darknet>에 접속하여 아래와 같이 다운로드 함
- 🕒 [Code] 버튼을 클릭하면 download 할 web URL을 클립보드에 복사하는 버튼이 있는데 클릭하여 복사함



- Colab 셀에 `!git clone` 다음에 클립보드에서 복사해온 URL을 Ctrl+V로 붙여넣기 하면 아래와 같은 URL 경로가 복사됨
- 여기서는 아래의 스크립트를 실행하면 됨

```
1 # darknet에서 yolov3.cfg/yolov3-tiny.cfg/coco.names 파일을 다운로드
2
3 # Colab에서 GitHub 에 있는 데이터 가져오기
4 # 다운로드 할 수 있는 URL : https://github.com/pjreddie/darknet Clone Web URL 를 복사해 옵니다.
5
6 !git clone https://github.com/pjreddie/darknet.git
```

```
Cloning into 'darknet'...
remote: Enumerating objects: 5955, done.
remote: Total 5955 (delta 0), reused 0 (delta 0), pack-reused 5955
Receiving objects: 100% (5955/5955), 6.37 MiB | 3.17 MiB/s, done.
Resolving deltas: 100% (3932/3932), done.
```

■ VM에 darknet 폴더가 있는지 확인

🔄 아래와 같이 darknet 폴더가 다운로드 된 것을 볼 수 있음

```
▶ # GitHub로부터 다운로드된 'darknet' dataset 폴더 확인  
  
!ls  
  
📁 darknet  gdrive  sample_data
```

■ VM에 darknet/cfg/ 폴더에 있는 파일 목록을 확인

🔄 아래와 같이 yolov3.cfg, yolov3-tiny.cfg 두 개의 신경망 구조 파일이 있는 것을 볼 수 있음

```
1 # darknet/cfg/ 폴더 파일 확인
2
3 !ls darknet/cfg/
```

alexnet.cfg	imagenet1k.data	vgg-16.cfg
ci far.cfg	imagenet22k.dataset	vgg-conv.cfg
ci far.test.cfg	imagenet9k.hierarchy.dataset	voc.data
coco.data	jnet-conv.cfg	writing.cfg
combine9k.data	openimages.data	yolo9000.cfg
darknet19_448.cfg	resnet101.cfg	yolov1.cfg
darknet19.cfg	resnet152.cfg	yolov1-tiny.cfg
darknet53_448.cfg	resnet18.cfg	yolov2.cfg
darknet53.cfg	resnet34.cfg	yolov2-tiny.cfg
darknet9000.cfg	resnet50.cfg	yolov2-tiny-voc.cfg
darknet.cfg	resnext101-32x4d.cfg	yolov2-voc.cfg
densenet201.cfg	resnext152-32x4d.cfg	yolov3.cfg
extraction22k.cfg	resnext50.cfg	yolov3-openimages.cfg
extraction.cfg	rnn.cfg	yolov3-spp.cfg
extraction.conv.cfg	rnn.train.cfg	yolov3-tiny.cfg
go.cfg	strided.cfg	yolov3-voc.cfg
go.test.cfg	t1.test.cfg	
gru.cfg	tiny.cfg	

■ VM에 darknet/data/ 폴더에 있는 파일 목록을 확인

🔄 아래와 같이 coco.names 부류 명칭이 들어 있는 파일이 있는 것을 볼 수 있음

```
1 # darknet/data/ 폴더 파일 확인
2
3 !ls darknet/data/
```

9k.labels	coco.names	goal.txt	inet9k.map	person.jpg
9k.names	dog.jpg	horses.jpg	kite.jpg	scream.jpg
9k.tree	eagle.jpg	imagenet.labels.list	labels	voc.names
coco9k.map	giraffe.jpg	imagenet.shortnames.list	openimages.names	

■ 내 구글 드라이브로 올로 신경망 구조, 부류 명칭 파일을 복사

🔄 darknet 폴더에서 `yolov3.cfg`, `yolov3-tiny.cfg`, `coco.names` 파일을 아래 위치로 복사

구글 드라이브 아래 `/CV/Object Detection_YOLO/config/`

```
1 # GitHub로부터 다운로드된 'darknet' dataset에서 필요한 파일만 복사 한다.  
2  
3 !cp darknet/cfg/yolov3.cfg gdrive/My Drive/CV/Object Detection_YOLO/config/  
4 print('yolov3.cfg copy complete!!')  
5  
6 !cp darknet/cfg/yolov3-tiny.cfg gdrive/My Drive/CV/Object Detection_YOLO/config/  
7 print('yolov3-tiny.cfg copy complete!!')  
8  
9 !cp darknet/data/coco.names gdrive/My Drive/CV/Object Detection_YOLO/config/  
10 print('coco.names copy complete!!')
```

```
📁 yolov3.cfg copy complete!!  
yolov3-tiny.cfg copy complete!!  
coco.names copy complete!!
```

■ 내 구글 드라이브 아래 /CV/Object Detection_YOLO/config/ 폴더의 파일목록을 확인

🔄 아래와 같이 옴로 모델에서 필요한 파일들이 복사된 것을 볼 수 있음

```
[ ] # 내 구글 드라이브의 gdrive/My Drive/CV/Object Detection_YOLO/config/ 폴더 파일 확인  
!  
!ls gdrive/My Drive/CV/Object Detection_YOLO/config  
  
coco.names yolov3.cfg yolov3-tiny.cfg yolov3-tiny.weights yolov3.weights
```

■ VM에서 다운로드 받은 darknet 폴더를 제거

- 🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨

```
[ ] # 다운로드 받았던 폴더(파일)를 제거한다.  
  
!rm -r darknet
```

- 🔄 VM에서 파일 목록을 확인함

```
[ ] # 내 구글 드라이브 확인  
  
!ls  
  
gdrive sample_data
```

참고

올로 모델에 필요한 파일 설명

YOLOv3은 MS COCO 데이터셋에 대해 학습이 되어 있음

coco.names

알고리즘이 감지할 수 있는 객체의 부류 이름이 포함된 파일

이 파일은 아래의 URL에 접속하면 화면에서 1~80줄로 부류 명칭을 확인할 수 있음

» URL : <https://github.com/pjreddie/darknet/blob/master/data/coco.names>

YOLOv3.weights

YOLOv3의 가중치를 저장한 파일

이 파일은 아래의 올로 공식 사이트에서 다운로드 받을 수 있음

» URL : <https://pjreddie.com/media/files/yolov3.weights>

yolov3.cfg

신경망 구조 정보가 들어 있는 파일

이 파일은 아래의 사이트에서 다운로드 받을 수 있음

» URL : <https://github.com/pjreddie/darknet/tree/master/cfg>

참고

올로 모델에 필요한 파일 설명

YOLOv3-tiny.weights YOLOv3-tiny의 **가중치**를 **저장**한 파일

🔄 이 파일은 아래의 사이트에서 다운로드 받을 수 있음

➡ URL : <https://pjreddie.com/media/files/yolov3-tiny.weights>

YOLOv3-tiny.cfg **신경망 구조 정보**가 들어 있는 파일

🔄 이 파일은 아래의 사이트에서 다운로드 받을 수 있음

➡ URL : <https://github.com/pjreddie/darknet/tree/master/cfg>

■ 이미지에서 객체 인식으로 인정할 최소 확률(신뢰도)을 50%로 지정

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨

```
# detection 으로 인정할 최소 확률(신뢰도) 지정  
# - 여기서 최소 확률(신뢰도)을 50%로 지정  
  
min_confidence = 0.5
```


YOLOv3.weights, YOLOv3.cfg

■ YOLOv3.weights, YOLOv3.cfg 파일을 불러와 YOLO 네트워크를 연결

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨

```
# Load YOLOv3
# 미리 학습된 덤러닝 파일을 OpenCV DNN 모듈로 YOLO 네트워크 연결
# 순전파(추론)만 가능하며 학습은 지원하지 않는다.

net = cv2.dnn.readNet("gdrive/My Drive/CV/Object Detection_YOLO/config/yolov3.weights",
                      "gdrive/My Drive/CV/Object Detection_YOLO/config/yolov3.cfg")
```

■ YOLOv3 네트워크의 모든 계층 이름을 출력

🔄 getLayerNames() 함수는 네트워크의 모든 레이어 이름을 가져옴

» YOLOv3 모델은 254개 계층으로 구성된 것을 알 수 있음



```
1 # YOLOv3 인공신경망 계층 이름을 출력
2
3 layer_names = net.getLayerNames()
4 print("layer_names len:", len(layer_names)) # layer_names len: 254
5 print("layer_names:", layer_names)         # layer_names: ('conv_0', 'bn_0', 'leaky_1', 'conv_1', ...
```

```
layer_names: ('conv_0', 'bn_0', 'leaky_1', 'conv_1', 'bn_1', 'leaky_2', 'conv_2', 'bn_2', 'leaky_3', 'conv_3', 'bn_3', 'leaky_4', 'shortcut_4', 'conv_5', 'bn_5', 'leaky_6', 'conv_6',
'bn_6', 'leaky_7', 'conv_7', 'bn_7', 'leaky_8', 'shortcut_8', 'conv_9', 'bn_9', 'leaky_10', 'conv_10', 'bn_10', 'leaky_11', 'shortcut_11', 'conv_12', 'bn_12', 'leaky_13', 'conv_13',
'bn_13', 'leaky_14', 'conv_14', 'bn_14', 'leaky_15', 'shortcut_15', 'conv_16', 'bn_16', 'leaky_17', 'conv_17', 'bn_17', 'leaky_18', 'shortcut_18', 'conv_19', 'bn_19', 'leaky_20',
'conv_20', 'bn_20', 'leaky_21', 'shortcut_21', 'conv_22', 'bn_22', 'leaky_23', 'conv_23', 'bn_23', 'leaky_24', 'shortcut_24', 'conv_25', 'bn_25', 'leaky_26', 'conv_26', 'bn_26',
'leaky_27', 'shortcut_27', 'conv_28', 'bn_28', 'leaky_29', 'conv_29', 'bn_29', 'leaky_30', 'shortcut_30', 'conv_31', 'bn_31', 'leaky_32', 'conv_32', 'bn_32', 'leaky_33',
'shortcut_33', 'conv_34', 'bn_34', 'leaky_35', 'conv_35', 'bn_35', 'leaky_36', 'shortcut_36', 'conv_37', 'bn_37', 'leaky_38', 'conv_38', 'bn_38', 'leaky_39', 'conv_39', 'bn_39',
'leaky_40', 'shortcut_40', 'conv_41', 'bn_41', 'leaky_42', 'conv_42', 'bn_42', 'leaky_43', 'shortcut_43', 'conv_44', 'bn_44', 'leaky_45', 'conv_45', 'bn_45', 'leaky_46',
'shortcut_46', 'conv_47', 'bn_47', 'leaky_48', 'conv_48', 'bn_48', 'leaky_49', 'shortcut_49', 'conv_50', 'bn_50', 'leaky_51', 'conv_51', 'bn_51', 'leaky_52', 'shortcut_52', 'conv_53',
'bn_53', 'leaky_54', 'conv_54', 'bn_54', 'leaky_55', 'shortcut_55', 'conv_56', 'bn_56', 'leaky_57', 'conv_57', 'bn_57', 'leaky_58', 'shortcut_58', 'conv_59', 'bn_59', 'leaky_60',
'conv_60', 'bn_60', 'leaky_61', 'shortcut_61', 'conv_62', 'bn_62', 'leaky_63', 'conv_63', 'bn_63', 'leaky_64', 'conv_64', 'bn_64', 'leaky_65', 'shortcut_65', 'conv_66', 'bn_66',
'leaky_67', 'conv_67', 'bn_67', 'leaky_68', 'shortcut_68', 'conv_69', 'bn_69', 'leaky_70', 'conv_70', 'bn_70', 'leaky_71', 'shortcut_71', 'conv_72', 'bn_72', 'leaky_73', 'conv_73',
'bn_73', 'leaky_74', 'shortcut_74', 'conv_75', 'bn_75', 'leaky_76', 'conv_76', 'bn_76', 'leaky_77', 'conv_77', 'bn_77', 'leaky_78', 'conv_78', 'bn_78', 'leaky_79', 'conv_79', 'bn_79',
'leaky_80', 'conv_80', 'bn_80', 'leaky_81', 'conv_81', 'permute_82', 'yolo_82', 'identity_83', 'conv_84', 'bn_84', 'leaky_85', 'upsample_85', 'concat_86', 'conv_87', 'bn_87',
'leaky_88', 'conv_88', 'bn_88', 'leaky_89', 'conv_89', 'bn_89', 'leaky_90', 'conv_90', 'bn_90', 'leaky_91', 'conv_91', 'bn_91', 'leaky_92', 'conv_92', 'bn_92', 'leaky_93', 'conv_93',
'permute_94', 'yolo_94', 'identity_95', 'conv_96', 'bn_96', 'leaky_97', 'upsample_97', 'concat_98', 'conv_99', 'bn_99', 'leaky_100', 'conv_100', 'bn_100', 'leaky_101', 'conv_101',
'bn_101', 'leaky_102', 'conv_102', 'bn_102', 'leaky_103', 'conv_103', 'bn_103', 'leaky_104', 'conv_104', 'bn_104', 'leaky_105', 'conv_105', 'permute_106', 'yolo_106')
```

■ YOLOv3 인공신경망 네트워크 레이어 중 출력 레이어를 확인

- 🕒 getUnconnectedOutLayers() 함수는 인공신경망 네트워크 레이어 중 출력 레이어의 인덱스를 가져옴
- YOLOv3 모델은 “yolo_82”, “yolo_94”, “yolo_106” 계층인 것을 알 수 있음

```
1 # getUnconnectedOutLayers() : 인공신경망 네트워크의 마지막 레이어로 인식되는 연결되어 있지 않은 출력 레이어의 인덱스를 제공하는 함수
2
3 # Get the names of output layers
4 # 인공신경망 네트워크의 마지막 레이어로 인식되는 연결되어 있지 않은 출력 레이어의 인덱스 확인
5 print("net.getUnconnectedOutLayers() len", len(net.getUnconnectedOutLayers())) # net.getUnconnectedOutLayers() len 3
6 print(net.getUnconnectedOutLayers()) # [200 227 254]
7
8 # 레이어 중 출력 레이어의 인덱스를 가져와서 출력한다.
9 print([layer_names[i-1] for i in net.getUnconnectedOutLayers()]) # ['yolo_82', 'yolo_94', 'yolo_106']
10
11 # 레이어 중 출력 레이어의 인덱스를 가져와서 output_layers에 넣는다.
12 output_layers = []
13 try:
14     output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]
15 except(Exception): # 모든 예러 처리
16     print("")
17
18 print("output_layers len:", len(output_layers)) # output_layers len: 3
19 print("output_layers:", output_layers) # output_layers: ['yolo_82', 'yolo_94', 'yolo_106']
```

■ MS COCO 데이터셋에 사용된 80가지 부류 이름을 읽어와서 배열로 저장

🔄 'coco.names' 파일에 들어 있는 클래스 이름의 공백을 제거하여 classes 리스트에 넣음

```
1 # detection 할 Object(Class) list 배열을 정의
2 classes = []
3
4 # 80개의 Object(class)를 구분할 수 있는 Object의 이름을 classes 배열에 넣어준다.
5 with open("gdrive/My Drive/CV/Object Detection_YOLO/config/coco.names", "r") as f:
6     # 클래스 이름의 공백을 제거하여 classes 리스트에 넣는다.
7     classes = [line.strip() for line in f.readlines()]
8 print("classes len:", len(classes))           # classes len: 80
9 print("classes:", classes)                   # classes: ['person', 'bicycle', 'car', 'motorbike', ...]
```

classes len: 80

classes: ['person', 'bicycle', 'car', 'motorbike', 'aeroplane', 'bus', 'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe', 'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard', 'tennis racket', 'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut', 'cake', 'chair', 'sofa', 'pottedplant', 'bed', 'diningtable', 'toilet', 'tvmonitor', 'laptop', 'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toothbrush']

🔄 strip() : whitespace(띄어쓰기, 탭, 엔터)를 없애는 것으로 중간에 끼어있는 것은 없어지지 않음

■ 80개의 부류를 구분하기 위해 난수를 이용해 서로 다른 색깔을 설정함

- 80개 부류를 서로 다른 색깔로 구분하기 위한 난수를 생성하여 colors 변수에 할당
- » 아래 코드에서 0~255 사이의 랜덤 한 수로 (80, 3) 크기 형상으로 반환한 것을 볼 수 있음

`np.random.uniform(low=0.0, high=1.0, size=None)` :
random 모듈 안에 정의되어 있는 두 수 사이의 랜덤 한 소수를 리턴 시켜주는 함수

```
1 # Object 마다 컬러를 하나씩 다르게 지정
2 colors = np.random.uniform(0, 255, size=(len(classes), 3))
3 print("colors shape:", colors.shape) # colors shape: (80, 3)
4 print("colors:", colors[0:5])
```

```
colors shape: (80, 3)
colors: [[235.01093963 208.07098746 208.44801734]
 [219.59544969 137.54002927  64.0908944 ]
 [103.78250542  61.70933444 112.32695409]
 [105.3773565  182.90243816 207.76884854]
 [ 70.79845131 145.8140631  132.33892235]]
```

■ 원본 이미지를 출력

```
1 # 원본이미지를 읽어 온다.  
2  
3 img = cv2.imread("gdrive/My Drive/CV/Object Detection_YOLO/image/Street.jpg")  
4 img = cv2.resize(img, None, fx=0.7, fy=0.7)  
5 height, width, channels = img.shape  
6 print("image.shape :", img.shape)      # image.shape : (431, 814, 3)  
7  
8 cv2.imshow(img)                        # 원본 이미지 출력
```



■ 이미지 데이터 전처리([0,255] → 픽셀에 0.00392 값을 곱) 수행

🔄 영상의 크기는 416 x 416으로 변환

```
1 # cv2.dnn.blobFromImage() : 네트워크, 입력 블록(blob) 만들기
2 # YOLOv3의 Detecting model 3가지(320×320, 416×416, 608×608)
3 # 이미지 데이터 전처리([0,255] → 픽셀에 0.00392 값을 곱해준다)
4
5 # 영상의 전처리
6 blob = cv2.dnn.blobFromImage(img, 0.00392, (416, 416), (0, 0, 0), True, crop=False)
7 print("image.shape :", img.shape)      # image.shape : (431, 814, 3)
8 print(img[:3, :3, :])
9 print("blob.shape :", blob.shape)      # blob.shape : (1, 3, 416, 416)
10 print(blob[:, :, :3, :3])
```

```
image.shape : (431, 814, 3)
[[[152 151 153]
  [145 146 147]
  [142 144 145]]

 [[152 150 152]
  [146 147 148]
  [142 144 146]]

 [[149 148 150]
  [145 146 147]
  [144 146 147]]]]
```



영상 전처리

```
blob.shape : (1, 3, 416, 416)
[[[[[0.588    0.56448  0.48608 ]
     [0.588    0.56448  0.48608 ]
     [0.58016  0.5684   0.49392 ]

     [[0.58016  0.55272  0.4704  ]
     [0.58016  0.55272  0.4704  ]
     [0.57624  0.56056  0.47432 ]

     [[0.58016  0.5488   0.46648002]
     [0.58408   0.5488   0.46648002]
     [0.57624  0.55272  0.4704   ]]]]]]
```

(431, 814, 3) → (1, 3, 416, 416)

형상이 3차원에서 4차원으로 변경

■ 코드에서 신경망에 테스트 영상을 입력하고, 출력(추론)을 받아 outs 객체에 저장

- ② outs 객체는 (507, 85), (2328, 85), (8112, 85)를 리스트에 담고 있음
- ② 여기서 85-차원의 벡터는 (x,y,w,h,o,p1,p2,...,p80)을 표현함
 - └ 앞의 4개(x,y,w,h)는 바운딩 박스 정보
 - └ o는 물체일 가능성
 - └ 이후 80개는 물체 부류 확률

```
1 # setInput() : 네트워크 입력 설정하기
2 net.setInput(blob) # 네트워크에 전처리된 영상 입력 설정
3
4 # forward() : 네트워크 순방향 실행(추론)
5 # - outs에 detection 된 Object들이 있다.
6 outs = net.forward(output_layers) # 네트워크 순방향 추론 (output_layers: ['yolo_82', 'yolo_94', 'yolo_106'])
7 print("np.array(outs).shape :", np.array(outs).shape) # np.array(outs).shape : (3,)
8 print("np.array(outs)[0].shape :", np.array(outs)[0].shape) # np.array(outs)[0].shape : (507, 85)
9 print("np.array(outs)[1].shape :", np.array(outs)[1].shape) # np.array(outs)[1].shape : (2028, 85)
10 print("np.array(outs)[2].shape :", np.array(outs)[2].shape) # np.array(outs)[2].shape : (8112, 85)
```

■ outs 객체는 0번째 인덱스의 값을 출력

- ↻ outs 객체는 (3,) 형상 즉 3개의 값이 저장된 것을 알 수 있음
- ↻ outs 객체의 0번째 인덱스 형상은 (507, 85)임



```
1 # outs 객체의 0번째 인덱스 값을 출력
2 # - 506행, 0~9열 행열 값을 출력
3 print("np.array(outs).shape :", np.array(outs).shape)      # np.array(outs).shape : (3,)
4 print("np.array(outs)[0].shape :", np.array(outs)[0].shape) # np.array(outs)[0].shape : (507, 85)
5 print(outs[0][506:507,0:10])
6 # [[9.7039163e-01 9.6014607e-01 8.1518787e-01 8.1216145e-01 2.1700609e-08 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00]]
```

■ 신뢰도가 50% 넘는 사물을 모음

- ➡ 최고 부류 확률이 0.5보다 큰 값의 인식한 사물의 클래스 아이디(class_ids), 신뢰도(confidences), 사물을 인식해서 그릴 상자(boxes) 정보를 각각 객체에 저장

```
1 # Showing informations on the screen
2
3 class_ids = [] # detection 한 Class id를 저장하는 배열 정의
4 confidences = [] # detection 한 Class 의 신뢰도(확률)을 저장하는 배열 정의
5 boxes = [] # detection 한 boxing 정보를 저장하는 배열 정의
6
7 for out in outs: # outs: (3,) ==> (507, 85) (2028, 85) (8112, 85)
8     for detection in out: # (507, 85) (2028, 85) (8112, 85)
9         scores = detection[5:]
10        class_id = np.argmax(scores) # detection 한 Class id : 80개 사물중 가장 높은 확률
11        confidence = scores[class_id] # detection 한 Class 의 신뢰도(확률)
12        if confidence > min_confidence: # 최소 확률(신뢰도)을 50%로 지정
13            # Object detected
14            center_x = int(detection[0] * width)
15            center_y = int(detection[1] * height)
16            w = int(detection[2] * width)
17            h = int(detection[3] * height)
18
19            # Rectangle coordinates
20            x = int(center_x - w / 2)
21            y = int(center_y - h / 2)
22
23            boxes.append([x, y, w, h]) # boxing 정보를 boxes 배열에 저장
24            confidences.append(float(confidence)) # 신뢰도(확률)을 confidences 배열에 저장
25            class_ids.append(class_id) # Class id 를 class_ids 배열에 저장
```

■ 선정된 바운딩 박스에 NMS를 적용하여 최종 선정된 Bbox 인덱스를 구함

- 여기서 NMS 임계값은 0.4로 설정
- NMS(Non-Maximum Suppression, 비최대 억제)는 이미지의 동일한 개체에 대한 상자 중 가장 높은 확률을 가진 상자를 선택
- » 실행 결과 9, 4, 26, 27, 1, 21, 19, 24 인덱스가 출력된 것을 알 수 있음

```
1 # apply non-max suppression
2
3 # 비최대 억제(Non-Maximum Suppression, NMS) 알고리즘을 적용해 주위 바운딩 박스에 비해 최대를 유지한 것 인덱스 확인한다.
4 # - 박스안에 박스(노이즈)를 하나로 만들어 준다.
5
6 # 이 값이 클수록 box가 많이 사라진다(즉, 조금날 겹쳐도 NMS로 둘 중 하나를 삭제하기 때문)
7 nms_threshold = 0.4
8
9 # 최종 선정된 Bbox 인덱스 결과(노이즈가 제거된 박스만 표시해 준다)
10 indexes = cv2.dnn.NMSBoxes(bboxes, confidences, min_confidence, nms_threshold)
11 print("indexes:", indexes)           # indexes: [ 9  4 26 27  1 21 19 24]
```

■ 사물 인식 결과 이미지를 출력

» 아래 결과에서 **버스는 94.4% 확률로 제대로 검출**하고 분류했음을 알 수 있음

```

1  # 결과 이미지 출력
2
3  font = cv2.FONT_HERSHEY_PLAIN          # 폰트 설정
4
5  for i in range(len(bboxes)):           # 최종 선별한 Bbox 개수 만큼 반복
6      if i in indexes:                   # 노이즈가 제거된 박스만 표시해 준다.
7          x, y, w, h = boxes[i]
8          label = "{}: {:.2f}".format(classes[class_ids[i]], confidences[i]*100) # Class 이름, 신뢰도(확률) 표시
9          #label = str(classes[class_ids[i]]) # Class 이름
10         text_confidence = str(classes[class_ids[i]])+' '+ '%.3f'%confidences[i] # Class 신뢰도(확률) 표시
11         print(i, text_confidence)
12         #print(i, label)
13         color = colors[i]
14         cv2.rectangle(img, (x, y), (x + w, y + h), color, 1)
15         cv2.rectangle(img, (x, y - 20), (x + w, y), color, -1)
16         cv2.putText(img, label, (x + 5, y - 5), font, 1, (255, 255, 255), 1)
17
18  cv2.imshow('img')

```

```

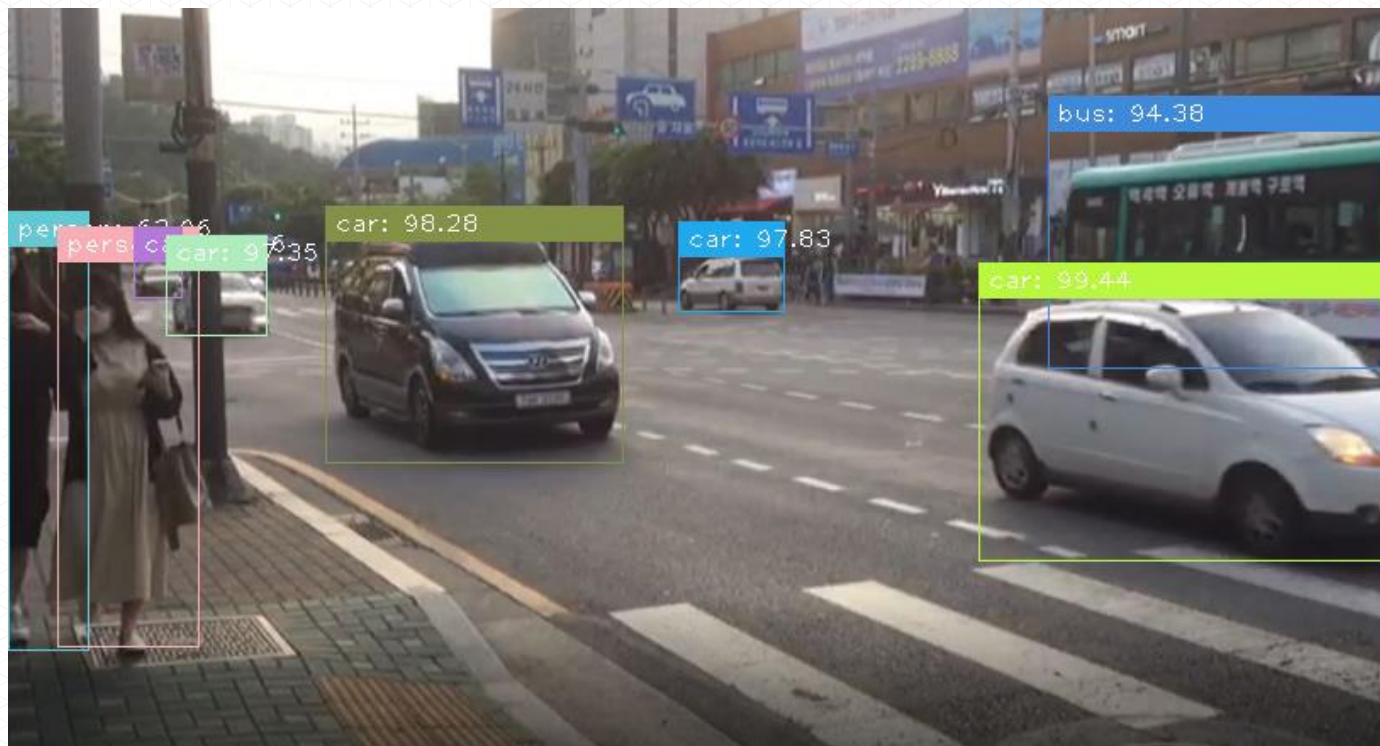
1 bus 0.944
4 car 0.983
9 car 0.994
19 person 0.630
21 person 0.916
24 car 0.543
26 car 0.978
27 car 0.974

```



■ 사물 인식 결과 이미지를 출력

1 bus 0.944
4 car 0.983
9 car 0.994
19 person 0.630
21 person 0.916
24 car 0.543
26 car 0.978
27 car 0.974



YOLOv3-tiny.weights, YOLOv3-tiny.cfg

■ YOLOv3-tiny.weights, YOLOv3-tiny.cfg 파일을 불러와 YOLO 네트워크를 연결

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨

```
1 # Load YOLOv3-tiny
2 # 미리 학습된 딥러닝 파일을 OpenCV DNN 모듈로 실행
3 # 순전파(추론)만 가능하며 학습은 지원하지 않는다.
4
5 net = cv2.dnn.readNet("gdrive/My Drive/CV/Object Detection_YOLO/config/yolov3-tiny.weights",
6                       "gdrive/My Drive/CV/Object Detection_YOLO/config/yolov3-tiny.cfg")
```


■ YOLO 네트워크의 모든 계층 이름을 출력

🔄 getLayerNames() 함수는 네트워크의 모든 레이어 이름을 가져옴

» YOLOv3-tiny 모델은 48개 계층으로 구성된 것을 알 수 있음

```
1 # YOLO 인공신경망 계층 이름을 출력
2
3 layer_names = net.getLayerNames()
4 print("layer_names len:", len(layer_names)) # layer_names len: 48
5 print("layer_names:", layer_names)         # layer_names: ('conv_0', 'bn_0', 'leaky_1', ...
```

```
layer_names: ('conv_0', 'bn_0', 'leaky_1', 'pool_1', 'conv_2', 'bn_2', 'leaky_3', 'pool_3', 'conv_4',
'bn_4', 'leaky_5', 'pool_5', 'conv_6', 'bn_6', 'leaky_7', 'pool_7', 'conv_8', 'bn_8', 'leaky_9', 'pool_9',
'conv_10', 'bn_10', 'leaky_11', 'pool_11', 'conv_12', 'bn_12', 'leaky_13', 'conv_13', 'bn_13', 'leaky_14',
'conv_14', 'bn_14', 'leaky_15', 'conv_15', 'permute_16', 'yolo_16', 'identity_17', 'conv_18', 'bn_18',
'leaky_19', 'upsample_19', 'concat_20', 'conv_21', 'bn_21', 'leaky_22', 'conv_22', 'permute_23',
'yolo_23')
```

■ YOLOv3-tiny 인공지능망 네트워크 레이어 중 출력 레이어를 확인

- 🔄 getUnconnectedOutLayers() 함수는 인공지능망 네트워크 레이어 중 출력 레이어의 인덱스를 가져옴
- » YOLOv3-tiny 모델은 “yolo_16”, “yolo_23” 계층인 것을 알 수 있음

```
1 # getUnconnectedOutLayers() : 인공지능망 네트워크의 마지막 레이어로 인식되는 연결되어 있지 않은 출력 레이어의 인덱스를
2
3 # Get the names of output layers
4
5 # 인공지능망 네트워크의 마지막 레이어로 인식되는 연결되어 있지 않은 출력 레이어의 인덱스 확인
6 print("net.getUnconnectedOutLayers() len", len(net.getUnconnectedOutLayers())) # net.getUnconnectedOutLayers() len 2
7 print(net.getUnconnectedOutLayers()) # [36 48]
8
9 # 레이어 중 출력 레이어의 인덱스를 가져와서 출력한다.
10 print([layer_names[i-1] for i in net.getUnconnectedOutLayers()]) # ['yolo_16', 'yolo_23']
11
12 # 레이어 중 출력 레이어의 인덱스를 가져와서 output_layers에 넣는다.
13 output_layers = []
14 try:
15     output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]
16 except (Exception): # 모든 예러 처리
17     print("")
18
19 print("output_layers len:", len(output_layers)) # output_layers len: 2
20 print("output_layers:", output_layers) # ['yolo_16', 'yolo_23']
```

■ MS COCO 데이터셋에 사용된 80가지 부류 이름을 읽어와서 배열로 저장

🕒 'coco.names' 파일에 들어 있는 클래스 이름의 공백을 제거하여 classes 리스트에 넣음

```
1 # detection 할 Object(Class) list 배열을 정의
2 classes = []
3
4 # 80개의 Object(class)를 구분할 수 있는 Object의 이름을 classes 배열에 넣어준다.
5 with open("gdrive/My Drive/CV/Object Detection_YOLO/config/coco.names", "r") as f:
6     # 클래스 이름의 공백을 제거하여 classes 리스트에 넣는다.
7     classes = [line.strip() for line in f.readlines()]
8 print("classes len:", len(classes))           # classes len: 80
9 print("classes:", classes)                   # classes: ['person', 'bicycle', 'car', 'motorbike', ...]
```

classes len: 80

classes: ['person', 'bicycle', 'car', 'motorbike', 'aeroplane', 'bus', 'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe', 'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard', 'tennis racket', 'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut', 'cake', 'chair', 'sofa', 'pottedplant', 'bed', 'diningtable', 'toilet', 'tvmonitor', 'laptop', 'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toothbrush']

🕒 strip() : whitespace(띄어쓰기, 탭, 엔터)를 없애는 것으로 중간에 끼어있는 것은 없어지지 않음

■ 80개의 부류를 구분하기 위해 난수를 이용해 서로 다른 색깔을 설정함

- 80개 부류를 서로 다른 색깔로 구분하기 위한 난수를 생성하여 colors 변수에 할당
- » 아래 코드에서 0~255 사이의 랜덤 한 수로 (80, 3) 크기 형상으로 반환한 것을 볼 수 있음

`np.random.uniform(low=0.0, high=1.0, size=None)` :
random 모듈 안에 정의되어 있는 두 수 사이의 랜덤 한 소수를 리턴 시켜주는 함수

```
1 # Object 마다 컬러를 하나씩 다르게 지정
2 colors = np.random.uniform(0, 255, size=(len(classes), 3))
3 print("colors shape:", colors.shape) # colors shape: (80, 3)
4 print("colors:", colors[0:5])
```

```
colors shape: (80, 3)
colors: [[235.01093963 208.07098746 208.44801734]
 [219.59544969 137.54002927  64.0908944 ]
 [103.78250542  61.70933444 112.32695409]
 [105.3773565  182.90243816 207.76884854]
 [ 70.79845131 145.8140631  132.33892235]]
```

■ 원본 이미지를 출력



```
1 # 원본이미지를 읽어 온다.  
2  
3 img = cv2.imread("gdrive/My Drive/CV/Object Detection_YOLO/image/Street.jpg")  
4 img = cv2.resize(img, None, fx=0.7, fy=0.7)  
5 height, width, channels = img.shape  
6 print("image.shape :", img.shape)      # image.shape : (431, 814, 3)  
7  
8 cv2.imshow(img)                        # 원본 이미지 출력
```



■ 이미지 데이터 전처리([0,255] → 픽셀에 0.00392 값을 곱) 수행

🔄 영상의 크기는 416 x 416으로 변환

```
1 # cv2.dnn.blobFromImage() : 네트워크, 입력 블록(blob) 만들기
2 # YOLOv3의 Detecting model 3가지(320×320, 416×416, 608×608)
3 # 이미지 데이터 전처리([0,255] → 픽셀에 0.00392 값을 곱해준다)
4
5 # 영상의 전처리
6 blob = cv2.dnn.blobFromImage(img, 0.00392, (416, 416), (0, 0, 0), True, crop=False)
7 print("image.shape :", img.shape)      # image.shape : (431, 814, 3)
8 print(img[:3, :3, :])
9 print("blob.shape :", blob.shape)      # blob.shape : (1, 3, 416, 416)
10 print(blob[:, :, :3, :3])
```

```
image.shape : (431, 814, 3)
[[[152 151 153]
  [145 146 147]
  [142 144 145]]

 [[152 150 152]
  [146 147 148]
  [142 144 146]]

 [[149 148 150]
  [145 146 147]
  [144 146 147]]]]
```



영상 전처리

```
blob.shape : (1, 3, 416, 416)
[[[[[0.588    0.56448  0.48608 ]
     [0.588    0.56448  0.48608 ]
     [0.58016  0.5684   0.49392 ]

     [[0.58016  0.55272  0.4704  ]
     [0.58016  0.55272  0.4704  ]
     [0.57624  0.56056  0.47432 ]

     [[0.58016  0.5488   0.46648002]
     [0.58408   0.5488   0.46648002]
     [0.57624  0.55272  0.4704   ]]]]]]
```

(431, 814, 3) → (1, 3, 416, 416)

형상이 3차원에서 4차원으로 변경

■ 코드에서 신경망에 테스트 영상을 입력하고, 출력(추론)을 받아 outs 객체에 저장

- ② outs 객체는 (507, 85), (2328, 85)를 리스트에 담고 있음
- ② 여기서 85-차원의 벡터는 (x,y,w,h,o,p1,p2,...,p80)을 표현함
 - ↳ 앞의 4개(x,y,w,h)는 바운딩 박스 정보
 - ↳ o는 물체일 가능성
 - ↳ 이후 80개는 물체 부류 확률

```
1 # setInput() : 네트워크 입력 설정하기
2 net.setInput(blob) # 네트워크에 전처리된 영상 입력 설정
3
4 # forward() : 네트워크 순방향 실행(추론)
5 # - outs 에 detection 된 Object 들이 들어 있다.
6 outs = net.forward(output_layers) # 네트워크 순방향 추론(output_layers: ['yolo_16', 'yolo_23'])
7 print("np.array(outs).shape :", np.array(outs).shape) # (2,)
8 print("np.array(outs)[0].shape :", np.array(outs)[0].shape) # (507, 85)
9 print("np.array(outs)[1].shape :", np.array(outs)[1].shape) # (2028, 85)
```

■ outs 객체는 0번째 인덱스의 값을 출력

- ↻ outs 객체는 (2,) 형상 즉 2개의 값이 저장된 것을 알 수 있음
- ↻ outs 객체의 0번째 인덱스 형상은 (507, 85)임

```
1 # outs 객체의 0번째 인덱스 값을 출력
2 # - 506행, 0~9열 행열 값을 출력
3 print("np.array(outs).shape :", np.array(outs).shape)      # np.array(outs).shape : (2,)
4 print("np.array(outs)[0].shape :", np.array(outs)[0].shape) # np.array(outs)[0].shape : (507, 85)
5 print(outs[0][506:507,0:10])
6 # [[9.6677363e-01 9.5592797e-01 7.5573349e-01 6.5449971e-01 3.1039008e-08 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00]]
```

■ 신뢰도가 50% 넘는 사물을 모음

- 최고 부류 확률이 0.5보다 큰 값의 인식한 사물의 클래스 아이디(class_ids), 신뢰도(confidences), 사물을 인식해서 그릴 상자(boxes) 정보를 각각 객체에 저장

```
1 # Showing informations on the screen
2
3 class_ids = [] # detection 한 Class id를 저장하는 배열 정의
4 confidences = [] # detection 한 Class 의 신뢰도(확률)을 저장하는 배열 정의
5 boxes = [] # detection 한 boxing 정보를 저장하는 배열 정의
6
7 for out in outs: # outs: (3,) ==> (507, 85) (2028, 85) (8112, 85)
8     for detection in out: # (507, 85) (2028, 85) (8112, 85)
9         scores = detection[5:]
10        class_id = np.argmax(scores) # detection 한 Class id : 80개 사물중 가장 높은 확률
11        confidence = scores[class_id] # detection 한 Class 의 신뢰도(확률)
12        if confidence > min_confidence: # 최소 확률(신뢰도)을 50%로 지정
13            # Object detected
14            center_x = int(detection[0] * width)
15            center_y = int(detection[1] * height)
16            w = int(detection[2] * width)
17            h = int(detection[3] * height)
18
19            # Rectangle coordinates
20            x = int(center_x - w / 2)
21            y = int(center_y - h / 2)
22
23            boxes.append([x, y, w, h]) # boxing 정보를 boxes 배열에 저장
24            confidences.append(float(confidence)) # 신뢰도(확률)을 confidences 배열에 저장
25            class_ids.append(class_id) # Class id 를 class_ids 배열에 저장
```

■ 선정된 바운딩 박스에 NMS를 적용하여 최종 선정된 Bbox 인덱스를 구함

- 여기서 NMS 임계값은 0.4로 설정
- NMS(Non-Maximum Suppression, 비최대 억제)는 이미지의 동일한 개체에 대한 상자 중 가장 높은 확률을 가진 상자를 선택
- » 실행 결과 4, 1, 5, 7, 0 인덱스가 출력된 것을 알 수 있음

```
1 # apply non-max suppression
2
3 # 비최대 억제(Non-Maximum Suppression, NMS) 알고리즘을 적용해 주위 바운딩 박스에 비해 최대를 유지한 것 인덱스 확인한다.
4 # - 박스안에 박스(노이즈)를 하나로 만들어 준다.
5
6 # 이 값이 클수록 box가 많이 사라진다(즉, 조금만 겹쳐도 NMS로 둘 중 하나를 삭제하기 때문)
7 nms_threshold = 0.4
8
9 # 최종 선정된 Bbox 인덱스 결과(노이즈가 제거된 박스만 표시해 준다)
10 indexes = cv2.dnn.NMSBoxes(bboxes, confidences, min_confidence, nms_threshold)
11 print("indexes:", indexes)           # indexes: [4 1 5 7 0]
```

■ 사물 인식 결과 이미지를 출력

» 아래 결과에서 **사람은 전혀 인식하지 못한 것**을 볼 수 있음

```

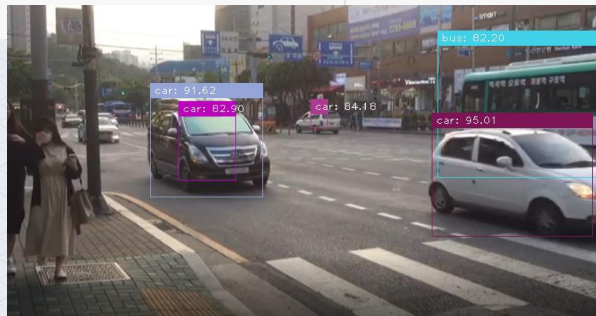
1 # 결과 이미지 출력
2
3 font = cv2.FONT_HERSHEY_PLAIN           # 폰트 설정
4
5 for i in range(len(boxes)):              # 최종 선별한 Bbox 개수 만큼 반복
6     if i in indexes:                     # 노이즈가 제거된 박스만 표시해 준다.
7         x, y, w, h = boxes[i]
8         label = "{}: {:.2f}".format(classes[class_ids[i]], confidences[i]*100) # Class 이름, 신뢰도(확률) 표시
9         #label = str(classes[class_ids[i]]) # Class 이름
10        text_confidence = str(classes[class_ids[i]])+' '+ '%.3f'%confidences[i] # Class 신뢰도(확률) 표시
11        print(i, text_confidence)
12        #print(i, label)
13        color = colors[i]
14        cv2.rectangle(img, (x, y), (x + w, y + h), color, 1)
15        cv2.rectangle(img, (x, y - 20), (x + w, y), color, -1)
16        cv2.putText(img, label, (x + 5, y - 5), font, 1, (255, 255, 255), 1)
17
18 cv2.imshow(img)

```

```

0 bus 0.822
1 car 0.916
4 car 0.950
5 car 0.842
7 car 0.829

```



■ 사물 인식 결과 이미지를 출력

0 bus 0.822
1 car 0.916
4 car 0.950
5 car 0.842
7 car 0.829

